Theseus Midterm Presentation Script

分工

- **A**：开场、项目进度、系统架构、当前实现铺垫、挑战与 AI 使用、收尾。
- **B**：完整负责 live demo，包括 frontend 页面展示、backend/API/DB proof。

目标时长：**10–12 分钟**

Slide 01 — Cover

Speaker: A
Time: 0:00–0:45

Good afternoon. We are Team Zisyphuz, and our project is **Theseus**.

Theseus is a weekly AI review system for goal-time alignment.

In simple words, it helps users look back at a week and compare three things: what they planned to do, what they actually spent time on, and what they should adjust for the next week.

For today’s midterm presentation, we will focus on the current implementation and the live demo. We will not spend much time repeating the original proposal background.

Right now, Sprint 1 and Sprint 2 are complete. The backend review engine and persistence path are working locally, and the Sprint 3 frontend branch is already ready for demo.

Slide 02 — Progress

Speaker: A
Time: 0:45–2:00

This slide shows our overall project progress.

The main point is that the backend and review engine are ahead of the original plan.

Sprint 0 was the foundation stage. We set up the project structure and clarified the main review workflow.

Sprint 1 focused on persistence. We added the local data model and SQLite storage path.

Sprint 2 focused on the review engine. This part is already working. The system can take weekly data and generate structured review output.

Sprint 3 is the frontend MVP. This part is still an active branch, but it is already strong enough for today’s demo.

So our current status is: backend and review engine are working locally, frontend demo branch is active, and final integration and evaluation are still coming next.

Slide 03 — App Preview

Speaker: A
Time: 2:00–2:50

This slide gives a quick preview of the frontend branch.

The key point here is that the frontend branch is ready for demo.

It is not the final merged version yet, so we are being careful with the wording. But it already shows the main user flow.

There are four main sections in the app: Review, Signals, Track, and Plan.

Review is where the user sees the weekly summary.

Signals gives quick status indicators, like plan, stage, goal, and energy.

Track shows the focus timer and time-tracking interaction.

Plan helps the user move from the review into the next week’s action.

We will not explain every screen here in detail, because B will show the actual flow in the live demo.

Slide 04 — Demo Plan

Speaker: A
Time: 2:50–3:30

This slide shows how the live demo will be structured.

The demo has three short frontend flows and then a backend proof section.

First, B will show the review flow, which is the main weekly review page.

Second, B will show the signals flow, where the app gives a compact view of the week's status.

Third, B will show the tracking flow, including the focus timer interaction.

After that, B will switch to backend proof. This includes the local API health check and the persisted review script.

One thing to mention before the demo: the frontend branch has fallback demo data. This is mainly for demo stability. If the backend is not running during the UI walkthrough, the frontend can still show the flow. But the backend is still proven separately through API and database output.

Now B will take over for the full demo.

Slide 04 Demo — Frontend Review Flow

Speaker: B
Time: 3:30–4:30

I will start with the frontend branch.

This is the current demo branch, not the final merged main branch yet.

The first screen I want to show is the Review page.

This page is the main weekly summary. It shows the user what happened during the week, what went well, what risks appeared, and what the next step should be.

The goal is not to create a long report. The goal is to give the user a quick weekly review that is easy to act on.

For example, instead of just saying "you worked a lot," the system tries to connect work back to goals and time logs.

So the Review page is the main output of the product.

Slide 04 Demo — Signals Flow

Speaker: B
Time: 4:30–5:15

Next, I will show the Signals section.

This page gives a more compact view of the current week.

It shows signals like plan, stage, goal, and energy status.

We added this because the review output can be easier to understand if the user first sees a few simple indicators.

So instead of reading everything at once, the user can quickly understand the state of the week.

This is meant to support the weekly review, not replace it.

Slide 04 Demo — Tracking Flow

Speaker: B
Time: 5:15–6:00

Now I will show the Track section.

This part includes the focus timer interaction.

The reason we added this is that Theseus is not only about goals. It is about the relationship between goals and actual time.

So the tracking flow helps connect planned work with real focus blocks.

For example, if the user planned to spend time on a project, the timer gives a simple way to record that work session.

This is still an MVP interaction, but it shows how the app connects review and time tracking.

Slide 04 Demo — Planning Flow

Speaker: B
Time: 6:00–6:40

The last frontend section is Plan.

This page gives a simple next-week cue.

The idea is that after the review, the user should not stop at reflection. They should get a clear action for the next week.

So the Plan page helps turn review output into a concrete restart point.

The frontend flow is basically:

Review, then Signals, then Track, then Plan.

That is the app-level demo.

Slide 06 — Backend and Implementation Proof

Speaker: B
Time: 6:40–8:30

Now I will switch from the frontend demo to backend proof.

First, I will check the local backend health endpoint.

```
GET /health
```

The response shows that the FastAPI backend is running locally.

Then I will show the persisted review script.

```
python scripts/run_persisted_review.py
```

This script takes sample weekly data, runs the review logic, generates structured review output, and stores the result in SQLite.

The important database table here is:

```
weekly_reviews
```

So this proves that the review output is not only displayed as mock data. The backend can generate and persist a real weekly review record locally.

To summarize the implementation proof:

The backend review and frontend demo are both repeatable locally. The frontend branch is ready for demo, and the backend review path is shown through API and database proof.

Slide 05 — Architecture

Speaker: A
Time: 8:30–9:30

Now I will connect the demo back to the system architecture.

The review pipeline is deterministic.

The basic flow is:

Input, API, SQLite, rules, and review.

The input includes goals, weekly plans, projects, and time logs.

The FastAPI backend validates and processes the data.

SQLite stores the local records, including weekly review output.

Then the review engine applies rule-based logic. It checks evidence such as wins, risks, drift, dormancy, and next steps.

Finally, the system produces structured review output.

The important part is that this is not an autonomous AI agent. The current review engine is rule-based and evidence-driven.

AI may help with wording and implementation support, but the actual review claims are grounded in deterministic logic and stored data.

Slide 07 — GitHub / Evidence

Speaker: A
Time: 9:30–10:10

This slide shows implementation evidence.

We have backend work, frontend branch work, scripts, and tests organized in the codebase.

The main point is that this is no longer only a proposal or design idea.

We now have a local backend path, a review engine, SQLite persistence, and an active frontend branch.

The frontend is still not final main, so we are not overstating it. But it is ready to support the midterm demo.

Slide 08 — Challenges and AI Usage

Speaker: A
Time: 10:10–11:20

Now I will talk about challenges and AI usage.

The first challenge is scope clarity.

The frontend is an active branch, not the final production version. So we are careful to present it as a demo branch.

The second challenge is integration.

The frontend can use fallback data for demo stability. That means the live UI demo can still work even if the backend is not running at that moment.

But we still show backend proof separately through API and database output.

The third challenge is AI wording.

Right now, the review engine is deterministic and rule-based. We are not claiming that an LLM is making final decisions.

AI was used as an implementation assistant. It helped with scaffolding routes, reviewing API structure, checking edge cases, writing tests, and improving documentation.

But AI is not the source of truth.

The source of truth is the implemented code, the scripts, the API behavior, the database output, and our manual verification.

So our boundary is simple:

AI assists. The team verifies.

Slide 09 — Closing

Speaker: A
Time: 11:20–12:00

To summarize, Theseus has moved from proposal scope into a working local review system.

At midterm, we have backend review logic working locally, SQLite persistence for weekly review output, and a frontend branch ready for demo.

The main thing we still need to finish is final integration, evaluation, and polish.

Our next steps are to complete the frontend integration, test the review quality with more examples, collect feedback, and prepare for the final presentation.

Thank you. We are ready for questions.

Backup Q&A

Q: Is the frontend fully connected to the backend?

Not fully yet. The frontend branch is ready for demo and can use fallback data for stability. The backend is shown separately through the health endpoint and persisted review script.

Q: Is the review generated by an LLM?

Not in the current core engine. The review engine today is deterministic and rule-based. AI helped with implementation and wording, but the review logic is verified through code, scripts, and manual checks.

Q: What is already working?

The local backend health endpoint works, the rule-based review engine works, weekly review output can be persisted into SQLite, and the frontend branch can show the current app flow.

Q: What is still unfinished?

Final frontend-backend integration, evaluation, polish, and final presentation preparation are still in progress.

Q: Why use fallback data?

Fallback data keeps the live demo stable. It lets us show the frontend flow even if the backend connection has an issue during presentation. The backend is still proven separately.